

Deepfake Detection GAN (DDGAN) - Complete Documentation

Table of Contents

- 1. [Overview](#)
 - 2. [Architecture](#)
 - [DCT Feature Extractor](#)
 - [Discriminator](#)
 - [Generator](#)
 - 3. [Training Pipeline](#)
 - [Data Flow](#)
 - [Loss Functions](#)
 - [Training Loop](#)
 - 4. [Evaluation](#)
 - 5. [Usage Guide](#)
 - 6. [Configuration Reference](#)
-

Overview

DDGAN is a GAN-based deepfake detection system that leverages **Discrete Cosine Transform (DCT)** frequency analysis combined with adversarial training to create a robust deepfake detector. The key insight is that deepfake generation methods leave characteristic artifacts in the frequency domain that are often invisible to the naked eye but detectable by neural networks.

Key Features

- **Frequency-Domain Analysis:** Uses DCT to extract frequency features where deepfake artifacts are more prominent
- **Adversarial Robustness:** Generator creates perturbations to make the discriminator more robust against adversarial attacks
- **Transfer Learning:** ConvNeXt-Tiny backbone pretrained on ImageNet provides strong feature extraction
- **PyTorch Lightning:** Scalable, clean training with automatic mixed precision and checkpointing

Project Structure

```
DDGAN/  
├── train.py           # Main training script  
├── evaluate.py        # Evaluation script  
├── config.py          # Configuration dataclasses  
├── pyproject.toml     # Dependencies  
├── src/  
│   ├── data/  
│   │   └── dataset.py # HuggingFace dataset loading
```

```
├── transforms.py      # Image preprocessing & augmentation
├── models/
│   ├── dct_extractor.py # DCT feature extraction
│   ├── discriminator.py # DCT-enhanced discriminator
│   └── generator.py     # U-Net perturbation generator
├── training/
│   ├── trainer.py      # Lightning training module
│   └── losses.py       # Loss functions
├── utils/
│   ├── metrics.py      # Evaluation metrics
│   └── visualization.py # Plotting utilities
├── checkpoints/        # Saved model checkpoints
└── logs/               # Training logs (CSV)
```

Architecture

DCT Feature Extractor

File: `src/models/dct_extractor.py`

The DCT (Discrete Cosine Transform) converts images from spatial domain to frequency domain, similar to JPEG compression. Deepfake generation methods often leave artifacts in specific frequency bands.

DCT2D Module

```
Input: Image tensor [B, C, H, W]
Output: DCT coefficients [B, C, H, W]
```

The 2D DCT is computed as:
$$F(u,v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos\left[\frac{\pi(2x+1)u}{2N}\right] \cos\left[\frac{\pi(2y+1)v}{2N}\right]$$

Where:

- $f(x,y)$ is the pixel value at position (x,y)
- $F(u,v)$ is the DCT coefficient at frequency (u,v)
- $\alpha(u) = \sqrt{1/N}$ if $u=0$, else $\sqrt{2/N}$

DCTFeatureExtractor

Extracts multi-scale DCT features by:

1. Computing DCT on the full image
2. Extracting low, mid, and high frequency bands
3. Computing statistics (mean, std, max) per band
4. Concatenating into a feature vector

```
# Feature dimensions
low_freq: [B, C, H//4, W//4] # DC and low frequencies
mid_freq: [B, C, H//2, W//2] # Mid-range frequencies
high_freq: [B, C, H, W] # High frequencies (edges, noise)
```

Why DCT for Deepfakes?

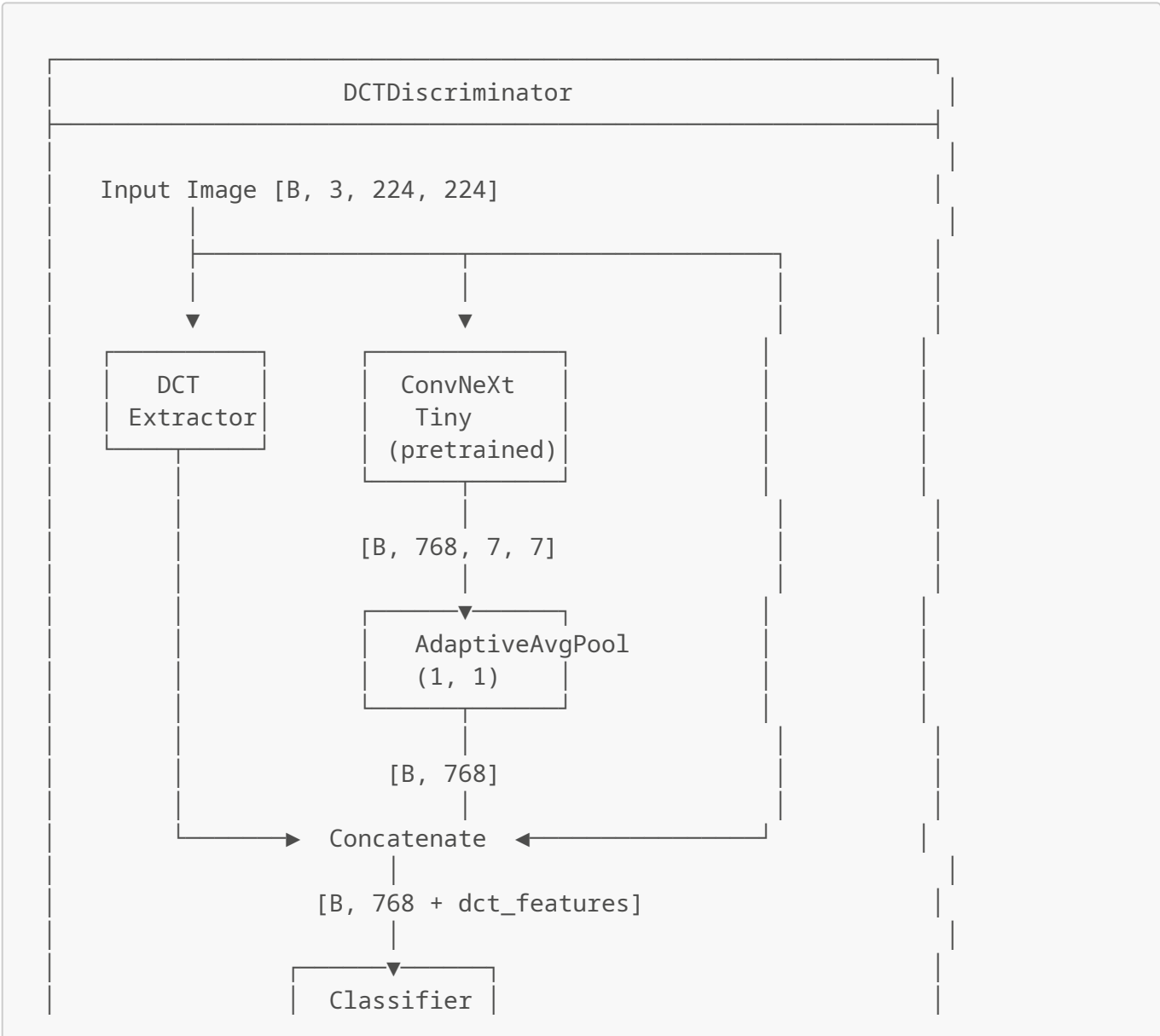
- GAN-generated images often have unnatural high-frequency patterns
- Face-swapping creates discontinuities at blend boundaries
- Compression artifacts differ between real and synthetic images

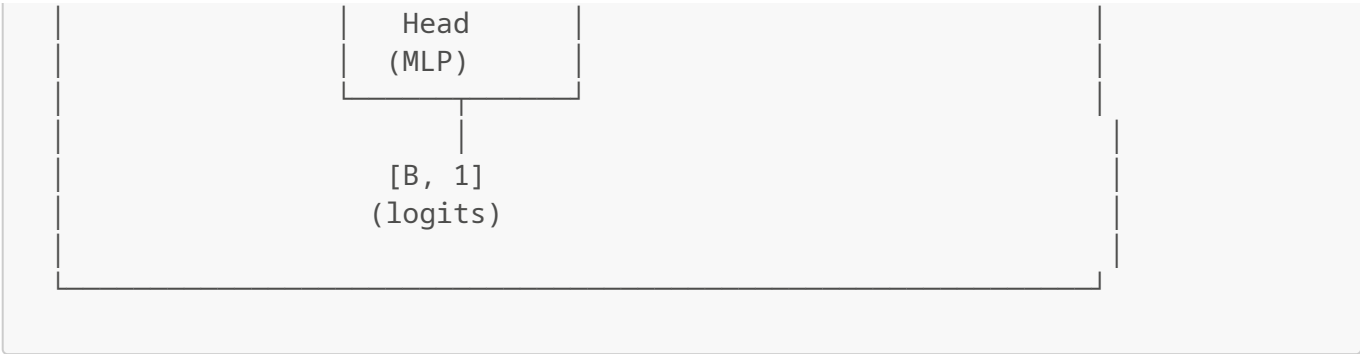
Discriminator

File: `src/models/discriminator.py`

The discriminator classifies images as real (1) or fake (0) using both spatial and frequency features.

Architecture Diagram





Classifier Head

```
Sequential(
  Linear(768 + dct_dim, 512),
  GELU(),
  Dropout(0.3),
  Linear(512, 256),
  GELU(),
  Dropout(0.3),
  Linear(256, 1) # Binary classification logit
)
```

Key Components

- **ConvNeXt-Tiny Backbone:** Pretrained on ImageNet-1K, provides powerful spatial feature extraction
- **DCT Branch:** Captures frequency-domain artifacts
- **Feature Fusion:** Concatenates spatial and frequency features
- **Dropout Regularization:** Prevents overfitting (p=0.3)

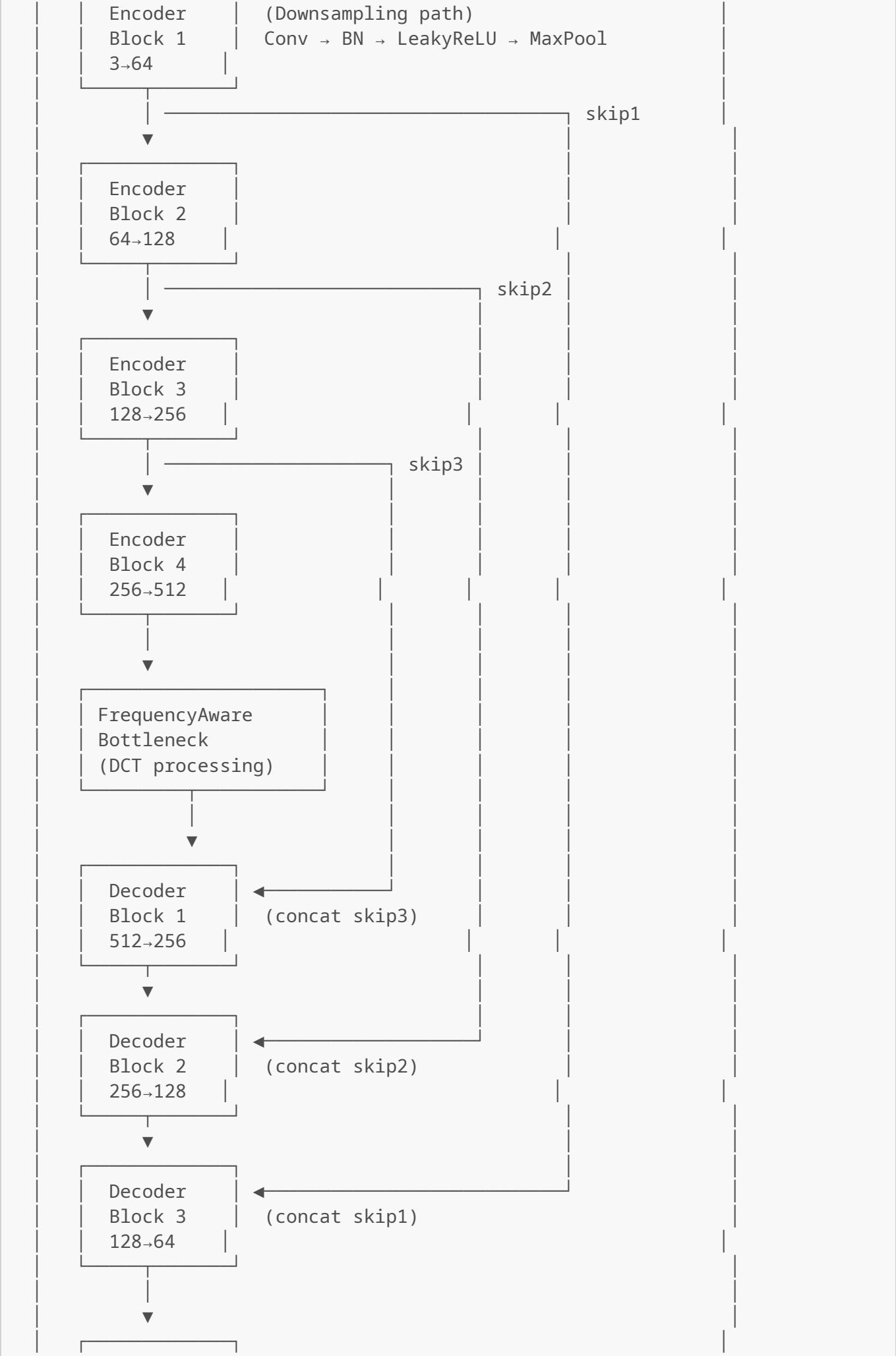
Generator

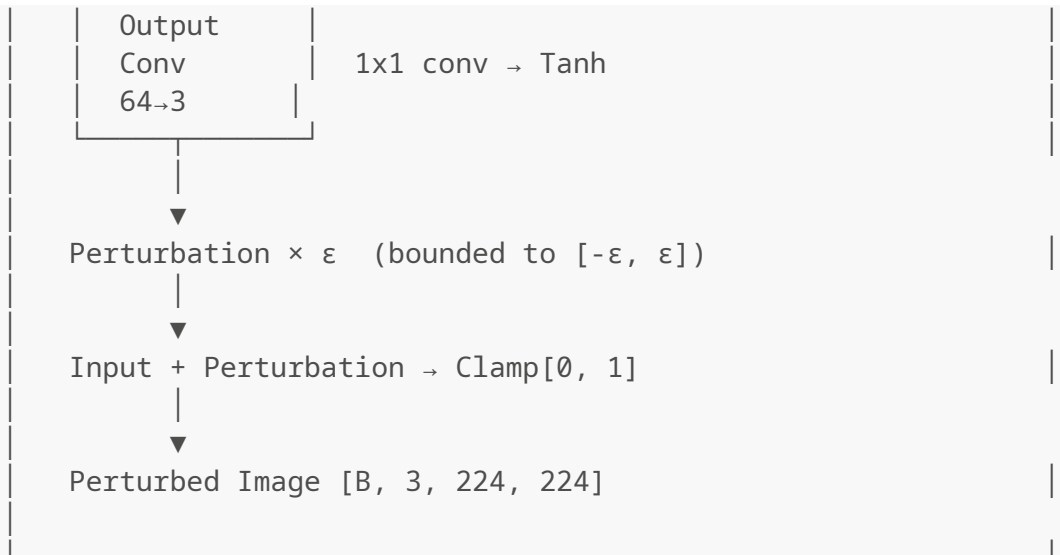
File: `src/models/generator.py`

The generator creates adversarial perturbations to make images harder for the discriminator to classify correctly. This trains the discriminator to be robust against adversarial attacks.

Architecture Diagram







FrequencyAwareBottleneck

The bottleneck processes features in both spatial and frequency domains:

```

# Spatial path
spatial_out = conv_layers(x)

# Frequency path
x_dct = dct_2d(x)
freq_out = freq_conv(x_dct)
freq_out = idct_2d(freq_out)

# Fusion
output = spatial_out + freq_out
  
```

Perturbation Bounds

The output perturbation is bounded by epsilon (ϵ):

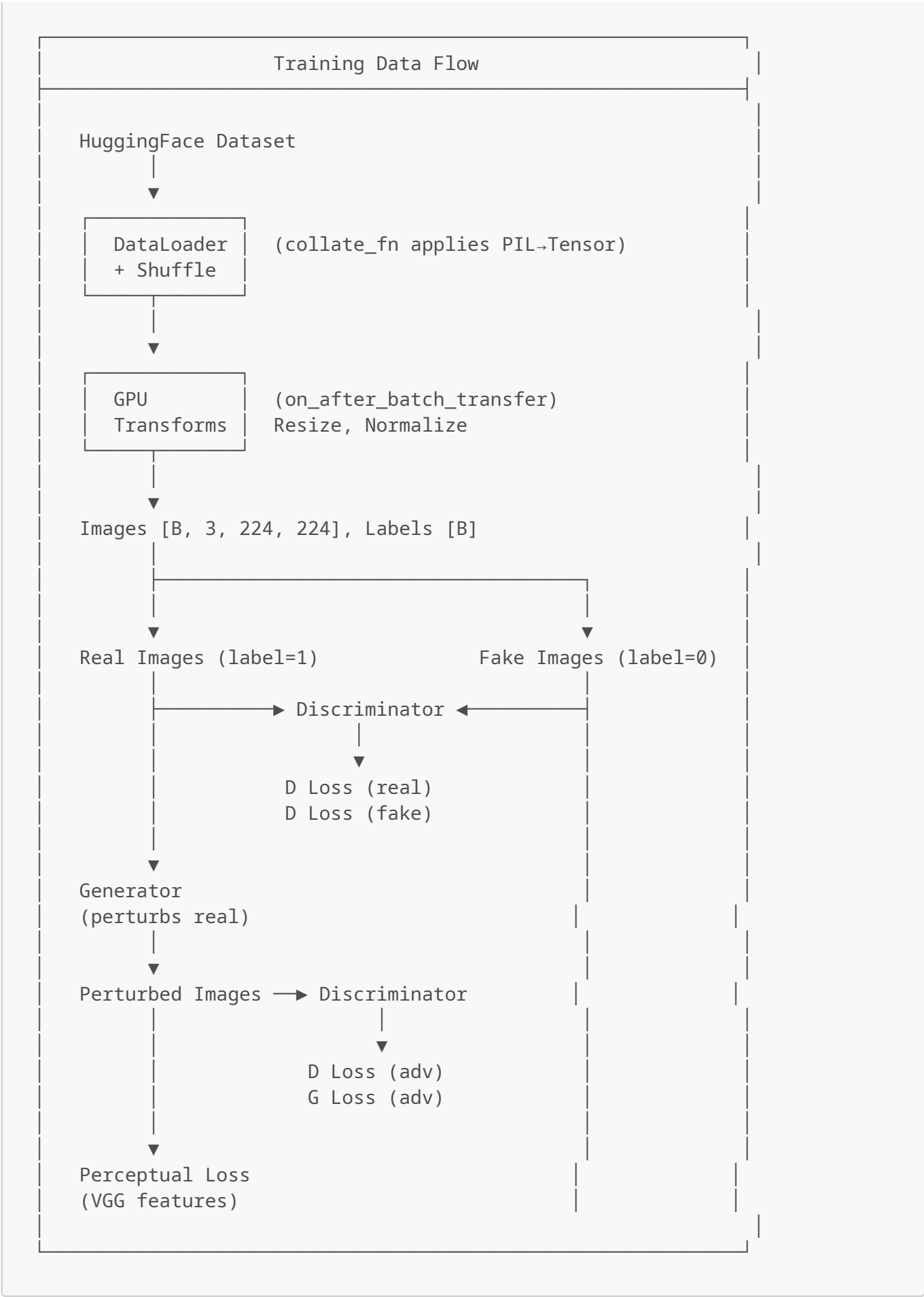
```

perturbation = self.epsilon * torch.tanh(raw_output)
perturbed_image = torch.clamp(image + perturbation, 0, 1)
  
```

Default $\epsilon = 0.03$ (~7.65 pixel values out of 255), making perturbations nearly imperceptible.

Training Pipeline

Data Flow



Loss Functions

File: [src/training/losses.py](#)

1. Binary Cross-Entropy with Logits (Discriminator)

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\sigma(x_i)) + (1-y_i) \log(1-\sigma(x_i))]$$

Where σ is the sigmoid function.

2. Adversarial Loss (Generator)

The generator tries to fool the discriminator: $\mathcal{L}_{adv} = \mathbb{E}[\log(1 - D(G(x)))]$

In practice, we use the "non-saturating" variant for better gradients: $\mathcal{L}_{adv} = -\mathbb{E}[\log(D(G(x)))]$

3. Perceptual Loss

Uses pretrained VGG-19 features to ensure perturbations don't drastically change image content:

$$\mathcal{L}_{perceptual} = \sum_l ||\phi_l(x) - \phi_l(G(x))||_2^2$$

Where ϕ_l is the feature map at layer l of VGG-19.

4. Combined Generator Loss

$$\mathcal{L}_G = \lambda_{adv} \cdot \mathcal{L}_{adv} + \lambda_{perceptual} \cdot \mathcal{L}_{perceptual}$$

Default weights: $\lambda_{adv} = 1.0$, $\lambda_{perceptual} = 0.1$

Training Loop

File: `src/training/trainer.py`

Each training step follows this procedure:

Training Step

1. DISCRIMINATOR UPDATE

Split batch: real_images, fake_images

Forward pass on real images

d_loss_real = BCE(D(real), ones)

Forward pass on fake images

d_loss_fake = BCE(D(fake), zeros)

Generate perturbations (G frozen)

perturbed = G(real)

Forward pass on perturbed (treated as fake)

d_loss_adv = BCE(D(perturbed), zeros)


```
└─ Total D loss:
    └─ d_loss = d_loss_real + d_loss_fake + d_loss_adv
└─ Backward + optimizer step (D only)

2. GENERATOR UPDATE
└─ Generate perturbations (D frozen)
    └─ perturbed = G(real)

└─ Adversarial loss (fool discriminator)
    └─ g_loss_adv = BCE(D(perturbed), zeros)
        (want D to predict "fake" as close to real)

└─ Perceptual loss (preserve content)
    └─ g_loss_perceptual = VGG_diff(real, perturbed)

└─ Total G loss:
    └─ g_loss = g_loss_adv + 0.1 * g_loss_perceptual
└─ Backward + optimizer step (G only)

3. LOG METRICS
    └─ d_loss, g_loss, accuracies, etc.
```

Optimization Details

- **Optimizer:** AdamW with weight decay 0.01
- **Learning Rate:** 2e-4 for both D and G
- **Scheduler:** CosineAnnealingLR (decays to 1e-6)
- **Gradient Clipping:** Max norm 1.0
- **Mixed Precision:** FP16 (16-mixed) for faster training

Evaluation

File: `evaluate.py`

Metrics Computed

Metric	Formula	Description
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Overall correct predictions
Precision	$\frac{TP}{TP + FP}$	Of predicted real, how many are actually real
Recall	$\frac{TP}{TP + FN}$	Of actual real, how many were detected
F1 Score	$\frac{2 \cdot P \cdot R}{P + R}$	Harmonic mean of precision and recall
ROC-AUC	Area under ROC curve	Performance across all thresholds

Where:

- TP = True Positives (real correctly classified as real)
- TN = True Negatives (fake correctly classified as fake)
- FP = False Positives (fake incorrectly classified as real)
- FN = False Negatives (real incorrectly classified as fake)

Evaluation Pipeline

1. Load checkpoint (supports both Lightning and legacy formats)
2. Load evaluation dataset
3. Forward pass on all samples (no gradients)
4. Compute predictions and scores
5. Calculate metrics
6. Generate visualizations:
 - Confusion matrix
 - ROC curve
 - DCT feature visualization
 - Misclassified examples
7. Save results to JSON and text report

Output Files

```
evaluation_results/  
├─ classification_report.txt    # Human-readable report  
├─ metrics.json                # Machine-readable metrics  
├─ confusion_matrix.png        # Confusion matrix visualization  
├─ roc_curve.png               # ROC curve plot  
├─ dct_features.png            # DCT feature visualization  
└─ misclassified_examples.png  # Examples of errors
```

Usage Guide

Installation

```
# Clone repository  
git clone <repo-url>  
cd DDGAN  
  
# Install dependencies  
pip install -e .  
  
# Or install manually  
pip install torch torchvision lightning datasets pillow matplotlib scikit-learn tqdm
```

Training

Basic Training

```
python train.py --dataset_name "your-dataset/name" --epochs 50
```

Full Options

```
python train.py \  
  --dataset_name "RohanRamesh/genimage-224" \  
  --batch_size 32 \  
  --epochs 50 \  
  --d_lr 2e-4 \  
  --g_lr 2e-4 \  
  --precision "16-mixed" \  
  --epsilon 0.03 \  
  --checkpoint_dir "checkpoints" \  
  --log_dir "logs" \  
  --val_to_train 0.5 \  
  --test_to_train 0.0 \  
  --refresh_rate 50 \  
  --seed 42
```

Resume Training

```
python train.py --dataset_name "..." --resume checkpoints/last.ckpt
```

Google Colab

```
!pip install -e .  
!python train.py --dataset_name "RohanRamesh/genimage-224" --epochs 50 --  
num_workers 2
```

Kaggle

```
!pip install -e .  
!python train.py --dataset_name "RohanRamesh/genimage-224" --epochs 50 --  
refresh_rate 50
```

Evaluation

```
python evaluate.py \  
  --checkpoint checkpoints/best-epoch=XX-val_accuracy=0.XXXX.ckpt \  
  --dataset_name "RohanRamesh/genimage-224" \  
  --split val \  
  --batch_size 32 \  
  --output_dir evaluation_results
```

Inference (Single Image)

```
import torch  
from PIL import Image  
from torchvision import transforms  
from src.training.trainer import DeepfakeGANModule  
  
# Load model  
model = DeepfakeGANModule.load_from_checkpoint("checkpoints/best.ckpt")  
model.eval()  
discriminator = model.discriminator  
  
# Preprocess image  
transform = transforms.Compose([  
    transforms.Resize((224, 224)),  
    transforms.ToTensor(),  
    transforms.Normalize(mean=[0.485, 0.456, 0.406],  
                          std=[0.229, 0.224, 0.225])  
)  
  
image = Image.open("test_image.jpg").convert("RGB")  
input_tensor = transform(image).unsqueeze(0) # Add batch dimension  
  
# Predict  
with torch.no_grad():  
    logit = discriminator(input_tensor)  
    probability = torch.sigmoid(logit).item()  
    prediction = "REAL" if logit > 0 else "FAKE"  
  
print(f"Prediction: {prediction} (confidence: {probability:.2%})")
```

Configuration Reference

Dataset Config

Parameter	Default	Description
<code>dataset_name</code>	None	HuggingFace dataset identifier
<code>cache_dir</code>	None	Local cache directory

Parameter	Default	Description
num_workers	4	DataLoader workers

Training Config

Parameter	Default	Description
batch_size	32	Training batch size
epochs	50	Number of training epochs
d_lr	2e-4	Discriminator learning rate
g_lr	2e-4	Generator learning rate
precision	"16-mixed"	Training precision
max_grad_norm	1.0	Gradient clipping threshold
accumulate_grad_batches	1	Gradient accumulation steps

Model Config

Parameter	Default	Description
pretrained	True	Use ImageNet pretrained weights
epsilon	0.03	Perturbation bound
image_size	224	Input image size

Data Split Config

Parameter	Default	Description
val_to_train	0.0	Fraction of val data to move to train
test_to_train	0.0	Fraction of test data to move to train

Logging Config

Parameter	Default	Description
log_every_n_steps	10	Logging frequency
val_check_interval	1.0	Validation frequency (1.0 = every epoch)
refresh_rate	0	Progress bar refresh rate

Troubleshooting

Common Issues

1. Out of Memory (OOM)

- Reduce `batch_size`
- Use `precision="16-mixed"`
- Reduce `num_workers`

2. Slow Training

- Increase `num_workers`
- Use `precision="16-mixed"`
- Enable `persistent_workers`

3. No Progress Bar on Kaggle

- Use `--refresh_rate 50`

4. Dataset Download Issues

- Set `--cache_dir` to a writable location
- Check HuggingFace authentication

5. Checkpoint Loading Errors

- Ensure checkpoint format matches (Lightning vs legacy)
- Check PyTorch version compatibility

Performance Tips

1. **Data Loading:** Use `num_workers=4` or higher on multi-core systems
2. **Mixed Precision:** Always use `precision="16-mixed"` on modern GPUs
3. **Batch Size:** Larger batches (64, 128) often train faster but may need lower LR
4. **Early Stopping:** Monitors `val/accuracy` with `patience=10` by default

References

- ConvNeXt: [A ConvNet for the 2020s](#)
- DCT for Deepfakes: [Detecting Deepfakes via Discrete Cosine Transform](#)
- U-Net: [U-Net: Convolutional Networks for Biomedical Image Segmentation](#)
- PyTorch Lightning: [lightning.ai](#)